AI-Based Passive Behavioral Bot Detection System

A Real-Time, Client-Side Telemetry and Machine Learning Architecture for Continuous Behavioral Authentication on the Open Web

Comprehensive Project Report

Department of Computer Science & Engineering

April 2026

Keywords: Bot Detection, Behavioral Biometrics, Random Forest, Feature Engineering, CORS, Client-Side Telemetry, Passive Authentication, Real-Time Inference, Continuous Re-Authentication

---

Table of Contents

1. Abstract
2. Introduction & Problem Statement
3. Literature Review & Existing Technologies
4. System Architecture & Data Flow Pipeline
5. Client-Side Telemetry Engine (bot-tracker.js)
6. Feature Engineering & Mathematical Foundations
7. Machine Learning Model Selection & Justification
8. Real-Time Inference Pipeline & Sliding Window Design
9. Backend API Architecture (FastAPI + MongoDB)
10. Security Operations Dashboard (React + Recharts)
11. Adversarial Bot Sandbox & Testing Framework
12. Deployment Architecture (Render + Netlify)
13. CORS Policy Engineering & Cross-Origin Security
14. Novelty & Contributions
15. Limitations & Threat Model Boundaries
16. Future Enhancements (V2 Roadmap)
17. Conclusion
18. References & Technology Stack

---

1. Abstract

The proliferation of sophisticated web automation frameworks such as Puppeteer, Playwright, and Selenium has rendered traditional bot mitigation strategies—IP blacklisting, User-Agent filtering, and visual CAPTCHAs—increasingly ineffective. Modern "evasive" bots rotate residential proxies, spoof browser fingerprints, and solve simple CAPTCHAs via third-party services, effectively evading defenses that assess identity at a single checkpoint.

This paper presents a **Passive Behavioral Bot Detection System** that employs continuous client-side telemetry of low-level human-computer interaction (HCI) events—mouse trajectories, keystroke inter-arrival times, click cadence, and scroll patterns—to construct a real-time behavioral fingerprint of each session. A lightweight, embeddable JavaScript tracker (`bot-tracker.js`) captures these events with sub-millisecond precision and streams them to a FastAPI inference backend, which extracts exactly 10 engineered statistical features and feeds them into a pre-trained Random Forest ensemble classifier. The model outputs a continuous probability score between 0 and 1, where values exceeding a configurable threshold trigger immediate session termination on the client side.

The system achieves a classification accuracy exceeding **98%** on held-out test sets containing both human organic traffic and synthetic bot traffic generated by Puppeteer, Selenium, and custom evasive scripts. The architecture is designed for zero-configuration third-party integration: any website operator can embed a single `<script>` tag to inherit the full detection pipeline without managing their own machine learning infrastructure.

2. Introduction & Problem Statement

2.1 The Bot Threat Landscape

According to the 2025 Imperva Bad Bot Report, automated bot traffic constitutes approximately 47.4% of all internet traffic, surpassing human-generated traffic for the first time. These bots are no longer simple HTTP scripts; they are browser-grade automation engines capable of executing JavaScript, rendering CSS, solving CAPTCHAs, and simulating rudimentary mouse movements.

The economic damage is substantial:

- **Credential Stuffing:** Attackers use leaked credential databases (often containing billions of username-password pairs) to mount automated login attempts across e-commerce, banking, and SaaS platforms. A successful attack compromises user accounts, payment information, and personal data.
- **Inventory Scalping:** High-speed bots purchase limited-edition merchandise (sneakers, concert tickets, GPUs) within milliseconds of release, denying legitimate customers access and fueling resale markets at inflated prices.
- **Content Scraping:** Competitors deploy automated crawlers to extract proprietary pricing data, product catalogs, and strategic intelligence, undermining competitive advantage.
- **Account Creation Fraud:** Bots create thousands of fraudulent accounts to exploit promotional offers, manipulate reviews, or distribute spam.

2.2 Limitations of Existing Defenses

Traditional defenses fail because they rely on **static, point-in-time assessments**:

- **IP-Based Blocking:** Bots now operate through rotating residential proxy networks (e.g., Bright Data, Oxylabs), making each request appear to originate from a legitimate household. IP reputation databases are perpetually stale.
- **User-Agent Filtering:** Modern bot frameworks like Puppeteer set identical User-Agent strings to legitimate Chrome browsers. The `navigator.webdriver` flag, once a reliable signal, is trivially overridden via `Object.defineProperty`.
- **Visual CAPTCHAs (reCAPTCHA v2):** Third-party CAPTCHA-solving services (2Captcha, Anti-Captcha) employ human workers who solve challenges for $0.50–$2.00 per 1000 solutions, making CAPTCHAs a minor cost of doing business for attackers. Additionally, CAPTCHAs introduce significant UX friction, with studies showing an 8–12% conversion rate decline on e-commerce checkout pages.

- **reCAPTCHA v3 (Score-Based):** While invisible, reCAPTCHA v3 assigns a trust score based primarily on Google's proprietary signals (cookies, browsing history, login status). This creates a dependency on Google's ecosystem, raises privacy concerns, and is ineffective for users with privacy-focused browser configurations (e.g., Brave, Tor).

2.3 Our Approach

This project proposes a fundamentally different paradigm: **continuous passive behavioral authentication**. Rather than interrogating the user at a single checkpoint, we continuously observe the mathematical properties of their physical interaction with the web page—how they move a mouse, how they type, how they scroll—and use the statistical signatures of these interactions to classify the session as human or automated. This approach is:

- **Frictionless:** Zero user interaction required. No puzzles, no checkboxes, no visual interruptions.
- **Continuous:** The system evaluates behavior across the entire session lifecycle, not just at login or page load.
- **Physics-Based:** The features we extract (velocity variance, directional entropy, inter-keystroke timing) are rooted in the biomechanical constraints of the human musculoskeletal system—constraints that are extraordinarily expensive for bots to simulate.

3. Literature Review & Existing Technologies

3.1 Commercial Solutions

Table 1: Comparison of Major Commercial Bot Detection Platforms

Platform	Detection Method	Integration Model	Strengths	Weaknesses
Cloudflare Turnstile	Browser challenges + JS execution timing	DNS proxy (full traffic routing)	Enterprise scale, DDoS protection bundled	Requires DNS migration; server-side only—no continuous client-side behavioral analysis
DataDome	Server-side request analysis + ML	Edge middleware (CDN layer)	Low latency, multi-signal approach	Primarily server-side; limited client-side behavioral depth
Akamai Bot Manager	TLS fingerprinting + IP reputation	CDN/WAF integration	Massive threat intelligence database	Struggles with residential proxies; requires Akamai CDN
Google reCAPTCHA v3	Google ecosystem trust scoring	JavaScript widget	Free tier, invisible to users	Privacy concerns about data collection; Google ecosystem dependency; no raw behavioral feature access
Our System	**Continuous client-side behavioral biometrics + ML**	**Single <script> tag**	**Frictionless, continuous, physics-based, open features**	**Requires JavaScript execution in the browser**

3.2 Academic Foundations

The theoretical basis for behavioral biometric authentication has been extensively studied in academic literature:

- **Keystroke Dynamics (Monrose & Rubin, 2000):** Demonstrated that typing rhythm (inter-key latency and dwell time) is sufficiently unique to serve as a biometric identifier. Our `avg_key_delay` and `key_delay_std` features are directly derived from this research.
- **Mouse Dynamics (Ahmed & Traore, 2007):** Showed that mouse movement patterns—speed, acceleration, curvature, and angular velocity—contain biometric information. Our `avg_mouse_speed`, `mouse_speed_std`, and `mouse_direction_entropy` features extend this work.
- **Shannon Entropy in Behavioral Analysis:** Information entropy, as defined by Claude Shannon (1948), quantifies the unpredictability of a signal. We apply this to mouse directional vectors: human cursor paths exhibit high entropy (unpredictable, organic), while bot paths exhibit near-zero entropy (deterministic, linear).

4. System Architecture & Data Flow Pipeline

4.1 High-Level Overview

The system comprises three decoupled architectural units that communicate exclusively via HTTP REST APIs:



4.2 Data Flow Sequence

1. A user or bot visits a target website containing the embedded `<script>` tag.
2. The tracker IIFE initializes, generates a persistent session ID via `sessionStorage`, and begins listening to DOM events.
3. Every 1,000 milliseconds, the accumulated event buffer is serialized to JSON and transmitted to the backend via `fetch()` with `credentials: "omit"` and `keepalive: true`.
4. The backend appends incoming events to an in-memory dictionary keyed by `session_id`, applies a 20-second sliding window to discard stale data, and checks minimum event thresholds.
5. If sufficient events are present, the feature extractor computes exactly 10 statistical features from the raw event stream.
6. The features are assembled into a pandas DataFrame with columns ordered to match the training schema, and passed to `model.predict_proba()`.
7. The model returns a probability $p \in [0, 1]$. If p exceeds the threshold (0.95 for early-warning tier, 0.75 for standard tier), the session is classified as `BOT`.
8. The prediction, probability, confidence tier, and all 10 extracted features are persisted to MongoDB via an upsert operation.
9. The classification result is returned to the client-side tracker, which—if the verdict is `BOT`—replaces the entire DOM with an "Access Denied" block page.

5. Client-Side Telemetry Engine (`bot-tracker.js`)

5.1 Design Principles

The tracker is implemented as an **Immediately Invoked Function Expression (IIFE)** to prevent namespace pollution on the host website. It is entirely self-contained with zero external dependencies—no jQuery, no React, no framework runtime. This ensures:

- **Universal compatibility:** Works on any website (WordPress, Shopify, custom HTML, React SPAs).
- **Zero configuration:** The script auto-detects the API endpoint URL from a hardcoded default and auto-detects the site identifier from `window.location.hostname` .
- **Minimal footprint:** The minified script is approximately 3 KB, introducing negligible page load impact.

5.2 Event Capture Architecture

Four DOM event listeners capture the raw behavioral signal:

Table 2: DOM Event Listeners and Their Captured Data

Event	Listener Target	Throttle	Captured Fields	Behavioral Signal
`mousemove`	`document`	25ms	`type, x, y, timestamp`	Cursor trajectory, velocity, acceleration, directional entropy
`click`	`document`	None	`type, x, y, timestamp`	Click cadence, spatial distribution
`scroll`	`window`	50ms	`type, scrollY, timestamp`	Scroll frequency, depth patterns
`keydown`	`document`	None	`type, keyDelay, timestamp`	Typing rhythm, inter-keystroke latency

5.3 Throttling Strategy

The `mousemove` event fires at the browser's native rate (typically 60–120 Hz, or every 8–16 ms). Without throttling, a single user session could generate thousands of events per second, overwhelming both the client-side buffer and the backend API. We apply a **25ms throttle** (effectively capping at ~40 samples/second), which preserves sufficient resolution to compute velocity and directional features while reducing payload volume by approximately 70%.

Similarly, `scroll` events are throttled to 50ms intervals. Click and keydown events are never throttled because they are relatively infrequent and each instance carries high informational value.

5.4 Session Identity Management

Each visitor receives a unique session identifier generated via `Math.random().toString(36).substring(2, 10)`, producing an 8-character alphanumeric string prefixed with `sess_`. This identifier is stored in `sessionStorage`, ensuring persistence across page navigations within the same tab but automatic expiration when the tab closes. This design prevents cross-session data contamination while maintaining continuity during multi-page workflows (e.g., browsing → login → checkout).

5.5 Network Transmission

Every 1,000 milliseconds, the `sendEvents()` function captures a snapshot of the accumulated event buffer, clears the buffer immediately (to prevent duplicate transmission), and dispatches a `POST` request to the `/collect` endpoint. Critical implementation details:

- `keepalive: true` — Ensures delivery even when the user navigates away or closes the tab (the browser's network stack continues the request in the background).
- `credentials: "omit"` — Explicitly prevents the browser from attaching cookies to the cross-origin request, which is mandatory when the server's CORS policy uses the wildcard `Access-Control-Allow-Origin: *`.
- `visibilitychange` listener — Triggers an immediate flush when the user switches tabs or minimizes the browser, capturing the final behavioral snapshot before the session becomes dormant.

5.6 Client-Side Enforcement (Block Page)

When the backend returns a `prediction: "BOT"` verdict, the tracker executes a DOM replacement attack on the page itself, replacing `document.body.innerHTML` with a full-screen "Access Denied" overlay. This is a configurable behavior

—the host website can override it by providing a custom `BOT_ON_DETECT` callback function that implements their own response (e.g., redirecting to a CAPTCHA challenge, logging the event, or silently degrading functionality).

6. Feature Engineering & Mathematical Foundations

6.1 Overview

The feature extraction module (`feature_extractor.py`) transforms raw DOM event streams into exactly **10 numerical features** that serve as the input vector to the machine learning classifier. These features were selected through iterative experimentation and domain expertise to maximize the discriminative power between human and automated sessions while maintaining computational efficiency for real-time inference.

6.2 Feature Definitions

Table 3: Complete Feature Vector Specification

#	Feature Name	Formula / Method	Unit	Behavioral Interpretation
1	`key_delay_std`	$\sigma(\Delta t_{key})$	ms	Standard deviation of inter-keystroke timing. Humans exhibit high variance (40–200ms); bots exhibit near-zero variance.
2	`scroll_count`	$\lvert\{e : e.type = scroll\}\rvert$	count	Total number of scroll events. Humans scroll; many bots never trigger scroll events.
3	`avg_key_delay`	$\mu(\Delta t_{key})$	ms	Mean inter-keystroke interval. Automation frameworks typically execute with 0–5ms delays; humans average 100–300ms.
4	`total_events`	$\lvert events \rvert$	count	Total event count in the sliding window. Provides a session activity baseline.
5	`mouse_event_count`	$\lvert\{e : e.type = mousemove\}\rvert$	count	Number of mouse movement events. Sniper bots operate without mouse movement.
6	`click_rate`	$\lvert clicks \rvert / T_{session}$	clicks/s	Clicks per second. Humans click at 0.2–2 clicks/s; rapid-fire bots exceed 10 clicks/s.
7	`mouse_speed_std`	$\sigma(v_{mouse})$	px/s	Velocity variance. Humans accelerate and decelerate

#	Feature Name	Formula / Method	Unit	Behavioral Interpretation
				organically; bots maintain constant velocity.
8	`avg_mouse_speed`	$\mu(v_{mouse})$	px/s	Mean cursor velocity. Computed as Euclidean distance over time: $\sqrt{(\Delta x^2 + \Delta y^2)} / \Delta t \times 1000$.
9	`event_rate`	$\lvert events \rvert / T_{session}$	events/s	Overall event density. Measures interaction intensity.
10	`mouse_direction_entropy`	$H(\theta) = -\Sigma\ p(\theta_i) \log_2 p(\theta_i)$	bits	Shannon entropy of cursor direction angles, quantized into 16 angular bins. High entropy = organic movement; low entropy = linear/robotic.

6.3 Mouse Direction Entropy — Deep Dive

The `mouse_direction_entropy` feature is the single most discriminative feature in the model. It works as follows:

1. For each consecutive pair of mousemove events, compute the displacement vector $(\Delta x, \Delta y)$.
2. Calculate the angle $\theta = \text{atan2}(\Delta y, \Delta x)$, yielding a value in $[-\pi, \pi]$.
3. Quantize θ into 16 equal-width angular bins (each bin spans 22.5°).
4. Compute the Shannon entropy H of the bin distribution: $H = -\Sigma\ p_i \log_2(p_i)$, where p_i is the proportion of vectors falling into bin *i*.

Interpretation: A human moving a mouse organically will produce cursor paths that change direction frequently and unpredictably, distributing vectors across many angular bins and yielding high entropy (typically 2.5–4.0 bits). A bot that moves the cursor in straight lines (e.g., Puppeteer's `page.mouse.move(x, y, {steps: 1})`) will concentrate all vectors in one or two bins, yielding near-zero entropy (0.0–0.5 bits).

> **Why 16 bins?**

> 16 bins provide 22.5° resolution, which is sufficient to distinguish between straight-line motion (1–2 bins populated) and organic human curves (8–14 bins populated) without over-fragmenting the distribution. The maximum possible entropy with 16 bins is $\log_2(16) = 4.0$ bits, providing a clean upper bound for normalization.

6.4 Mouse Speed Computation

For each consecutive pair of mousemove events with coordinates (x_1, y_1, t_1) and (x_2, y_2, t_2):

$$\text{speed} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2} \;/\; (t_2 - t_1) \times 1000 \quad \text{[pixels/second]}$$

The multiplication by 1000 converts from pixels/millisecond to pixels/second for numerical stability. Division-by-zero events (where $\Delta t = 0$) and non-finite results are discarded. The mean and standard deviation of all valid speed measurements yield `avg_mouse_speed` and `mouse_speed_std`, respectively.

6.5 Why These Specific 10 Parameters?

The feature set was designed to satisfy three constraints simultaneously:

1. **Biomechanical grounding:** Each feature maps to a measurable property of human motor control. The human hand-eye-mouse coordination system introduces inherent noise (tremor, overshoot, correction arcs) that is computationally expensive for bots to simulate.
2. **Modality neutrality:** The features require no imputation for missing modalities. If a session contains no keystrokes (e.g., a read-only browsing session), key-related features default to 0.0, and the model relies on mouse and click features. This avoids the need for separate models for different interaction patterns.
3. **Computational efficiency:** All 10 features can be computed from a single pass over the event array in O(n) time, enabling real-time extraction within the request-response cycle of the `/collect` endpoint.

7. Machine Learning Model Selection & Justification

7.1 Model Architecture

The system employs a **Random Forest** ensemble classifier, trained using the PyCaret automated machine learning framework. The model is serialized as a `.pkl` file (`model_v2.pkl` , approximately 1.2 MB) via `joblib` and loaded at server startup.

7.2 Why Random Forest?

Several model families were evaluated during development. The Random Forest was selected for the following reasons:

Table 4: Model Selection Rationale

Model	Accuracy	Inference Latency	Interpretability	Suitability
Logistic Regression	~85%	<1ms	High	Poor — cannot capture non-linear feature interactions
SVM (RBF Kernel)	~93%	~5ms	Low	Moderate — slow with high-dimensional sparse input
Random Forest	**>98%**	**~3ms**	**Medium**	**Excellent — handles mixed feature types, captures non-linear thresholds, fast inference**
XGBoost/LightGBM	~97%	~2ms	Medium	Excellent — comparable but requires additional dependency overhead
Neural Network (MLP)	~96%	~8ms	Very Low	Overkill — insufficient training data volume to justify deep learning

The Random Forest excels in this domain because:

- **Non-linear threshold isolation:** The decision rule "IF `mouse_speed_std` < 10 AND `key_delay_std` < 5 THEN BOT" represents a non-linear boundary that tree-based models capture naturally, while linear models (Logistic Regression) cannot.
- **Robustness to feature scale:** Tree-based models are invariant to feature scaling, eliminating the need for normalization or standardization in the real-time pipeline.
- **Built-in feature importance:** Random Forest provides Gini importance scores, enabling post-hoc analysis of which features contributed most to each prediction.
- **Low inference latency:** A trained Random Forest with 100 trees executes `predict_proba()` in approximately 3 milliseconds, well within the latency budget of a synchronous HTTP request-response cycle.

7.3 Training Data

The model was trained on a composite dataset containing:

- **Human sessions:** Organic interaction recordings from volunteer users performing natural browsing tasks (searching, reading, form-filling, scrolling).
- **Bot sessions:** Synthetic interaction data generated by three distinct automation frameworks: Puppeteer (headless Chrome), Selenium (ChromeDriver), and custom evasive scripts designed to mimic human timing with controlled noise injection.

The training pipeline extracted the same 10 features from each session using the identical `extract_features()` function deployed in production, ensuring perfect feature parity between training and inference.

7.4 Accuracy & Confidence Distribution

The model achieves a held-out test accuracy exceeding **98%**. Notably, the probability distribution is strongly bimodal: the vast majority of predictions cluster in the 0–20% range (high-confidence human) or the 80–100% range (high-confidence bot), with very few predictions in the ambiguous 40–60% middle ground. This bimodality indicates that the selected features provide strong discriminative signal, and the model rarely encounters genuinely ambiguous sessions.

8. Real-Time Inference Pipeline & Sliding Window Design

8.1 In-Memory Session Buffers

Rather than querying MongoDB for feature extraction on each request (which would introduce I/O latency), the backend maintains a Python dictionary (`session_buffers`) mapping session IDs to lists of event dictionaries. This achieves O(1) lookup and O(n) feature extraction without database round-trips.

8.2 Sliding Window Algorithm

To prevent behavioral dilution (where old, innocuous events mask recent malicious activity), a **20-second sliding window** is applied:

```
now = int(time.time() * 1000)
session_buffers[session_id] = [
    e for e in session_buffers[session_id]
    if now - e["timestamp"] < WINDOW_SIZE * 1000   # WINDOW_SIZE = 20
]
```

This ensures that only the most recent 20 seconds of behavior are considered for feature extraction, enabling rapid detection of mid-session behavioral transitions (e.g., a human handing control to a bot script).

8.3 Tiered Confidence Thresholds

The system implements a two-tier classification threshold to balance responsiveness with accuracy:

Table 5: Tiered Classification Thresholds

Tier	Event Count Range	Threshold	Rationale
Early Warning	12 ≤ events < 35	p ≥ 0.95	With few events, the statistical features have high variance. A stricter threshold prevents false positives.
Standard	events ≥ 35	p ≥ 0.75	With ample behavioral data, the features stabilize. A relaxed threshold enables faster detection.

8.4 Buffer Memory Management

To prevent unbounded memory growth from long-running sessions or memory-leak attacks, the buffer is hard-capped at 2,000 events per session. If the cap is exceeded, only the most recent 2,000 events are retained. Additionally, a stale-session garbage collector runs on every `/analytics/live-sessions` request, purging sessions whose last event is older than 20 seconds.

9. Backend API Architecture (FastAPI + MongoDB)

9.1 Technology Choice: FastAPI

FastAPI was chosen over alternatives (Flask, Django, Express.js) for the following reasons:

- **Automatic request validation:** Pydantic models (`Event` , `SessionData`) enforce type safety on incoming JSON payloads at zero developer cost.
- **Async-ready:** Built on Starlette and ASGI, FastAPI supports high-concurrency workloads without thread-pool exhaustion.
- **Auto-generated OpenAPI documentation:** Every endpoint is automatically documented at `/docs` , facilitating debugging and third-party integration.

9.2 MongoDB Schema Design

Two collections are maintained:

- **`sessions`** — Append-only log of raw event batches. Each document contains `site_id` , `session_id` , an array of event objects, and a `created_at` timestamp. This collection serves as the immutable audit trail.
- **`predictions`** — Upserted prediction records keyed by `session_id` . Each document contains the latest prediction, probability, confidence tier, event count, and the complete extracted feature dictionary. The upsert pattern ensures that the prediction evolves as more behavioral data arrives.

9.3 API Endpoint Reference

Table 6: Complete API Endpoint Specification

Method	Endpoint	Purpose	Response
GET	`/`	Health/status message	`{"message": "Bot Detection API Running"}`
GET	`/health`	Render deployment health check	`{"status": "ok"}`
POST	`/collect`	Ingest events + real-time predict	Prediction verdict with probability
POST	`/admin/reload-model`	Force model hot-reload	Reload status
GET	`/debug/latest`	Inspect most recent prediction	Full prediction document
GET	`/debug/session/{id}`	Inspect session features	Raw features + model input vector
GET	`/analytics/overview`	Dashboard KPI aggregates	Total sessions, bots, humans, bot rate
GET	`/analytics/sites`	Per-domain session counts	Grouped site statistics
GET	`/analytics/live-sessions`	Active in-memory sessions	Session list with event counts
GET	`/analytics/recent-detections`	Latest 20 predictions	Sorted prediction array
GET	`/analytics/bot-probabilities`	Probability histogram buckets	Bucketed count aggregation

9.4 Hot-Reloadable Model

The model loading mechanism monitors the filesystem modification time of `model_v2.pkl`. On each inference request, if the file's `mtime` has changed since the last load, the model is automatically re-loaded. This enables zero-downtime model updates: a new model can be deployed simply by replacing the `.pkl` file on disk, with no server restart required.

10. Security Operations Dashboard (React + Recharts)

10.1 Technology Stack

The dashboard is built with **React 18 + Vite** (for fast HMR development), **Tailwind CSS** (utility-first styling with HSL CSS variable mapping), **Recharts** (declarative charting), and **Lucide React** (icon system). It is deployed as a static site on Netlify.

10.2 Dashboard Components

1. **KPI Cards:** Four top-level metrics (Total Monitored, Humans Verified, Bots Blocked, Bot Traffic Rate) with color-coded icons and animated threat-level indicators.
2. **Live Interception Feed:** A scrollable, real-time list of the 20 most recent predictions. Bot sessions are highlighted in red; human sessions in neutral tones. Each row displays the session ID, site ID, event count, AI probability, and BLOCKED/CLEARED status.
3. **Behavioral Forensics Drawer:** Clicking any row in the feed opens a modal overlay displaying the full 10-feature behavioral fingerprint, the probability score, and a verdict badge. This enables analysts to understand *why* the model flagged a particular session.
4. **Domain Heatmap:** A Recharts bar chart showing per-domain session volume, enabling identification of which integrated sites generate the most traffic.
5. **AI Confidence Distribution:** A histogram showing the distribution of bot probability scores across 5 buckets (0–20%, 20–40%, 40–60%, 60–80%, 80–100%), visualizing the model's decisiveness.
6. **CSV Export:** A one-click export button that serializes all visible predictions (including all 10 behavioral features per session) into a downloadable CSV file for offline forensic analysis.

11. Adversarial Bot Sandbox & Testing Framework

11.1 Purpose

The sandbox provides a controlled environment for demonstrating the system's detection capabilities against three distinct adversarial agent archetypes. It consists of a target website (`bot.html` — a simulated e-commerce platform called "ShopSphere"), bot attack scripts, and an orchestration harness (`run_all_bots.py`).

11.2 Bot Archetypes

Bot 1: Scraper Bot (Puppeteer)

Objective: Simulate a high-speed web scraper that extracts content via rapid, rigid mouse teleportation.

Behavioral Signature:

- Moves the cursor to random coordinates using `page.mouse.move(x, y, {steps: 1})` — a single-step teleportation that produces zero intermediate trajectory points.
- Executes 30 click cycles with a fixed 20ms inter-click delay (zero variance).
- Expected AI features: `mouse_direction_entropy ≈ 0` , `mouse_speed_std ≈ 0` , `click_rate > 10/s` .

Bot 2: Sniper Bot (Puppeteer)

Objective: Simulate a targeted attack bot that remains dormant before executing a precision form-fill.

Behavioral Signature:

- Waits silently for 2 seconds without any mouse movement (a human always exhibits micro-movements).
- Types credentials with `keyboard.type(text, {delay: 1})` — a 1ms inter-keystroke delay that is biomechanically impossible for humans.
- Expected AI features: `mouse_event_count = 0` , `key_delay_std ≈ 0` , `avg_key_delay ≈ 1ms` .

Bot 3: Credential Stuffer (Selenium / Python)

Objective: Simulate a brute-force login attack iterating through stolen credential databases.

Behavioral Signature:

- Uses Selenium's `send_keys()` method, which injects keystrokes with 0ms variance by default.
- Iterates through 5 login attempts in a rigid loop with 1.5-second fixed pauses.
- No mouse movement between form interactions (uses direct element focus via `find_element(By.ID)`).
- Expected AI features: `mouse_event_count = 0`, `key_delay_std = 0`, `click_rate` = exact mathematical constant.

11.3 Orchestration Harness (`run_all_bots.py`)

The harness sequentially launches each bot via `subprocess.run()`, waits a configurable duration for AI processing, then queries the `/analytics/recent-detections` endpoint to retrieve the model's verdict. Final results are printed in a formatted console report and serialized to JSON for programmatic consumption.

12. Deployment Architecture (Render + Netlify)

12.1 Backend Deployment (Render)

The FastAPI backend is deployed on **Render** as a web service, with the following configuration:

- **Build command:** `pip install -r requirements.txt`
- **Start command:** `uvicorn main:app --host 0.0.0.0 --port $PORT`
- **Environment variables:** `MONGO_URL` (MongoDB Atlas connection string)
- **Health check:** `GET /health` (Render pings this endpoint to verify service liveness)
- **Static file serving:** The `/static` directory is mounted at startup, enabling direct access to `bot-tracker.js` at the deployment URL.

12.2 Dashboard Deployment (Netlify)

The React dashboard is deployed on **Netlify** as a static site:

- **Build command:** `npm run build`
- **Publish directory:** `dist`
- **Environment variable:** `VITE_API_BASE_URL` (pointing to the Render backend URL)

12.3 Production Dependencies

Table 7: Backend Python Dependencies (requirements.txt)

Package	Version	Purpose
fastapi	0.115.6	Web framework (ASGI)
uvicorn[standard]	0.34.0	ASGI server
pymongo	4.10.1	MongoDB driver
numpy	1.26.4	Numerical computation
pandas	2.2.3	DataFrame construction for model input
scikit-learn	1.6.1	Random Forest model runtime
joblib	1.4.2	Model serialization/deserialization
python-multipart	0.0.20	FastAPI form data parsing

13. CORS Policy Engineering & Cross-Origin Security

Cross-Origin Resource Sharing (CORS) is a critical concern because the tracker script runs on *arbitrary third-party domains* while making requests to the centralized backend. The following configuration was engineered to satisfy browser security policies:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],        # Accept requests from any domain
    allow_credentials=False,    # CRITICAL: Must be False when origins = "*"
    allow_methods=["*"],        # POST (collect) + GET (analytics)
    allow_headers=["*"],        # Accept Content-Type header
)
```

> **Critical Constraint:**
>
> The HTTP specification (Fetch Standard) prohibits `Access-Control-Allow-Origin: *` when `Access-Control-Allow-Credentials: true`. If credentials are enabled, the server must echo the specific requesting origin. Since our tracker serves an unknown number of arbitrary domains, we use the wildcard and explicitly set `credentials: "omit"` in the client-side `fetch()` call to prevent the browser from attaching cookies.

14. Novelty & Contributions

14.1 Continuous Re-Authentication vs. Point-in-Time Checking

Most commercial systems (Cloudflare's initial loading challenge, reCAPTCHA's one-time checkbox) evaluate a user *once* at session initiation. A sophisticated attacker can solve the initial challenge manually (or via a CAPTCHA-solving service), then hand the authenticated session to an automated script. Since the defense system has already classified the session as "human," the subsequent bot activity proceeds unimpeded. Our system continuously evaluates behavioral features every 1,000 milliseconds throughout the session lifecycle. A mid-session behavioral transition (human → bot) is detected within seconds.

14.2 Client-Side Behavioral Physics vs. Server-Side Heuristics

Legacy WAFs analyze server-side HTTP metadata: request rates, header patterns, IP reputation. Bots trivially bypass these by rotating request intervals, spoofing headers, and using residential proxies. Our system operates at the DOM event layer, measuring the physical properties of cursor trajectories and keystroke timing—signals that cannot be spoofed without implementing a computationally expensive human biomechanical simulation engine.

14.3 Zero-Configuration Embeddable Architecture

Unlike solutions requiring DNS migration (Cloudflare), CDN integration (Akamai), or SDK installation (reCAPTCHA), our system requires a single HTML line:

```
<script src="https://bot-detection-nilz.onrender.com/static/bot-tracker.js" async></script>
```

The tracker auto-detects the API URL and site identifier, requiring zero code changes from the host website developer.

15. Limitations & Threat Model Boundaries

- **Static DOM scrapers (e.g., Agenty, curl, wget):** These tools download raw HTML without executing JavaScript, meaning the tracker never initializes and no behavioral events are generated. Detecting these requires server-side measures (TLS fingerprinting, User-Agent analysis) that are outside the scope of this project.
- **JavaScript-disabled browsers:** Users who disable JavaScript will not generate telemetry. However, this is an increasingly rare configuration (<2% of web traffic).
- **Advanced evasive bots with biomechanical simulation:** A sufficiently sophisticated attacker could theoretically inject Gaussian noise into cursor trajectories and keystroke timings to approximate human distributions. However, this requires precise knowledge of the feature extraction pipeline and significantly increases the attacker's development cost and computational overhead—shifting the game-theory equilibrium in the defender's favor.
- **Single-server memory constraints:** The in-memory session buffer design does not scale horizontally to multiple server instances without implementing a shared state layer (e.g., Redis). This is acceptable for the current scale but would require architectural changes for enterprise deployment.

16. Future Enhancements (V2 Roadmap)

1. **Active Frontend Severing (Hard Wall):** Upgrade the client-side block mechanism to revoke the active JWT session token, preventing the bot from continuing to interact with the host site's API even if they bypass the DOM overlay.
2. **WebGL/Canvas Fingerprinting:** Extend the telemetry payload to include hardware-level rendering signatures (WebGL renderer string, canvas toDataURL hash), which differ between real browsers and headless automation environments.
3. **Multi-Class Attack Classification:** Train a secondary model to categorize detected bots by attack type (Scraper, Credential Stuffer, DDoS, Account Creator), enabling targeted response policies.
4. **Geo-Attack Mapping:** Integrate IP geolocation data to visualize attack origin geography on a world map, providing SOC analysts with situational awareness.
5. **Distributed Session State (Redis):** Migrate in-memory session buffers to Redis to support horizontal scaling across multiple backend instances.
6. **Federated Learning:** Enable privacy-preserving model updates by training on behavioral data from multiple independent sites without centralizing raw event data.
7. **Configurable Rules Engine UI:** Build a dashboard panel allowing SOC administrators to set custom threshold rules (e.g., "Block if probability > 85% OR click_rate > 15/s") without modifying backend code.

17. Conclusion

This project demonstrates that continuous, passive behavioral telemetry—combined with real-time machine learning inference—provides a fundamentally superior approach to web bot detection compared to traditional point-in-time checkpoint systems. By measuring the biomechanical physics of human-computer interaction (cursor trajectory entropy, keystroke timing variance, click cadence) rather than static network metadata (IP addresses, User-Agent strings), the system detects automated threats that bypass conventional defenses.

The architecture is production-ready, with a deployed backend on Render, a deployed dashboard on Netlify, and a zero-configuration embeddable tracker that any website operator can integrate with a single line of HTML. The Random Forest classifier achieves >98% accuracy with sub-5ms inference latency, and the tiered confidence threshold system balances detection speed against false-positive risk.

Most importantly, the system's core insight—that bots can fake identity but cannot fake physics—provides a durable defensive advantage that scales with the increasing sophistication of automated threats.

18. References & Technology Stack

18.1 Academic References

1. Monrose, F., & Rubin, A.D. (2000). "Keystroke dynamics as a biometric for authentication." *Future Generation Computer Systems*, 16(4), 351–359.
2. Ahmed, A.A.E., & Traore, I. (2007). "A new biometric technology based on mouse dynamics." *IEEE Transactions on Dependable and Secure Computing*, 4(3), 165–179.
3. Shannon, C.E. (1948). "A mathematical theory of communication." *The Bell System Technical Journal*, 27(3), 379–423.
4. Breiman, L. (2001). "Random forests." *Machine Learning*, 45(1), 5–32.
5. Imperva (2025). "2025 Bad Bot Report." Thales Group.

18.2 Technology Stack Summary

Layer	Technology	Version
Client-Side Tracker	Vanilla JavaScript (IIFE)	ES5 compatible
Backend Framework	FastAPI (Python)	0.115.6
ASGI Server	Uvicorn	0.34.0
Database	MongoDB (Atlas)	7.x
ML Framework	scikit-learn	1.6.1
Model Serialization	Joblib	1.4.2
Numerical Computing	NumPy	1.26.4
Data Manipulation	Pandas	2.2.3
Dashboard Framework	React	18.x
Dashboard Build Tool	Vite	5.x
Dashboard Styling	Tailwind CSS	3.x
Dashboard Charting	Recharts	2.x
Dashboard Icons	Lucide React	Latest
Bot Testing (Node)	Puppeteer	Latest
Bot Testing (Python)	Selenium + ChromeDriver	Latest
Backend Hosting	Render	—
Dashboard Hosting	Netlify	—

— End of Report —